

NovaSMS

Compréhension des Modules Backend

20 modules NestJS · Rôle · Technologies utilisées et pourquoi · Endpoints · Mécanismes notables · Tests

Stage Sankofa Lab · Romuald KO · Document régénéré depuis le code réel du dépôt
Plateforme SaaS B2B Messagerie Multi-Canal — NestJS · PostgreSQL · Redis · React 19

Table des matières

1	Vue d'ensemble — organisation modulaire
2	Schéma de données — 26 modèles Prisma
3	Auth — authentification & 2FA
4	Account & API Keys — compte, équipe, accès API
5	Contacts & Segments — CRM et ciblage
6	Campagnes, Templates & Track — cœur métier
7	Automations & Queues — orchestration asynchrone
8	Mobile Money & Transactions — paiement
9	Analytics & Audit Logs — reporting et traçabilité
10	Webhooks, Public API, Mail & Providers — intégrations externes
11	Common & Prisma — éléments transverses
12	Synthèse — technologies par module et justification

1 — Vue d'ensemble — organisation modulaire

Le backend NestJS (apps/backend/src) est organisé en une vingtaine de modules par domaine métier, chacun regroupant un contrôleur (routes HTTP), un service (logique métier), des DTOs (contrats d'entrée/sortie) et, pour la majorité, une suite de tests unitaires Jest colocalisée. Le point d'entrée main.ts configure les éléments transverses à tous les modules : préfixe global /api, Helmet, CORS, ValidationPipe global, filtre d'exception global, interceptor multi-tenant, et le module BullMQ pour les files d'attente.

Module	Rôle en une phrase
auth	Inscription, connexion, JWT, 2FA, invitations d'équipe
account	Profil du compte, équipe, crédits, préférences
api-keys	Clés API publiques, permissions, quotas
contacts	CRM, import de masse, segments, conformité RGPD
segments	CRUD segments dynamiques (chevauche contacts)
campaigns	Création, envoi et A/B testing de campagnes SMS/Email
templates	Modèles de messages réutilisables
track	Pixel d'ouverture, clics trackés, désabonnement
automations	Workflows déclenchés par événements métier
queues	Processors et workers BullMQ transverses
mobile-money	Paieement Mobile Money ouest-africain
transactions	Paieement carte bancaire (Stripe)
analytics	Statistiques et rapports de performance
audit-logs	Journal d'audit des actions du compte
webhooks	Réception statuts fournisseurs + abonnements sortants
public-api	API REST v1 pour intégration tierce
mail	Emails transactionnels système
providers	Abstraction multi-fournisseurs (SMS/Email/WhatsApp/paiement/stockage/push)
common	Guards, filtres, décorateurs, utilitaires transverses
prisma	Accès base de données (service global)

2 — Schéma de données — 26 modèles Prisma

4

Le schéma prisma/schema.prisma (626 lignes, 38 migrations versionnées) définit 26 modèles. Account constitue la racine du modèle multi-tenant : toutes les entités métier lui sont rattachées par accountId avec suppression en cascade.

Modèle	Rôle
Account	Compte client (tenant) — solde crédit, 2FA, préférences
User	Membre d'équipe d'un Account, avec un rôle
Contact / Segment	Répertoire client et ciblage dynamique
Campaign / Send / CampaignImage / ARTestResult	Campagne, envoi individuel par contact, image, résultat A/B
Automation / WorkflowExecution	Workflow d'automatisation et ses exécutions
Transaction / Invoice	Paiement carte et facture PDF associée
MobileMoneyTransaction	Paiement Mobile Money (identifiant opérateur externe)
Template	Modèle de message réutilisable
AuditLog	Traçabilité des actions sensibles
Invitation	Invitation d'un collaborateur à rejoindre un compte
ImportReport / Job	Suivi des imports de contacts en masse
Analytic / EngagementHeatmap / ClickHeatmap	Agrégats analytiques par campagne
Consent	Traçabilité du consentement RGPD
ProviderConfig	Configuration des fournisseurs externes par compte
ApiKey / ApiKeyLog	Clé API publique et journal d'appels
NotificationPrefs / CreditUsage / WebhookSubscription	Préférences, historique crédit, abonnements webhooks

%, Enums métier

PhoneStatus (VALID/INVALID/UNVERIFIED), CampaignStatus (DRAFT/SCHEDULED/SENDING/SENT/CANCELLED/FAILED/AUTOMATION), SendStatus (PENDING/SENT/OPENED/CLICKED/BOUNCED/UNSUBSCRIBED), UserRole (Admin/Editor/Analyst), TransactionMethod (MobileMoney/Visa), TransactionStatus (Pending/Validated/Refused/Timeout), AutomationStatus (Active/Inactive/Draft), JobStatus, AnalyticAction.

3.1 Rôle

Le module auth (auth.service.ts, ~1250 lignes — le plus long du backend) gère l'intégralité du cycle de vie d'identification : inscription, vérification d'email, connexion, rafraîchissement de session, 2FA à deux facteurs (application TOTP ou SMS), réinitialisation de mot de passe, et invitations d'équipe.

3.2 Technologies utilisées et pourquoi

- **bcryptjs (12 rounds)** !' hachage irréversible des mots de passe ; choisi pour sa simplicité d'intégration Node pure (pas de compilation native) contrairement à bcrypt
- **speakeasy** !' génération et vérification de codes TOTP compatibles Google Authenticator (secret base32, fenêtre de tolérance de 2 pas) pour la 2FA applicative
- **@nestjs/jwt avec deux secrets distincts (JWT_ACCESS_SECRET / JWT_REFRESH_SECRET)** !' sépare le cycle de vie du token d'accès (court) de celui du rafraîchissement (long), limitant la fenêtre d'exploitation d'un vol de token
- **ioredis (JwtBlacklistService)** !' permet la révocation immédiate d'un token au logout, chose impossible avec un JWT stateless pur
- **zod** !' validation des DTOs d'inscription/reset (register.dto.ts, forgot-password.dto.ts) — choix plus récent que class-validator dans l'historique du projet

3.3 Endpoints principaux

Méthode	Route	Rôle
POST	/auth/register	Inscription d'un nouveau compte
POST	/auth/login	Connexion (limité à 10/min)
POST	/auth/verify-2fa	Validation du code 2FA après login
POST	/auth/refresh	Renouvellement du token d'accès
POST	/auth/logout	Déconnexion + blacklist du JWT
POST	/auth/generate-2fa-secret	Génération du secret TOTP + QR code
POST	/auth/enable-2fa / disable-2fa	Activation / désactivation de la 2FA
POST	/auth/forgot-password	Demande de réinitialisation (limité à 5/min)
GET	/auth/invitation	Consultation d'une invitation d'équipe

&™ Mécanismes notables

Verrouillage de compte après 5 tentatives échouées (blocage 15 minutes)
Connexion supportant deux chemins distincts : compte principal (Account) ou membre invité (User)
Codes de secours 2FA (10 codes format XXXX-XXXX) générés à l'activation

Tests : 6 fichiers spec (auth.controller, auth.service, auth.service.session, jwt-auth.guard, jwt-blacklist.service, jwt.strategy) — environ 76 tests unitaires, le module le mieux couvert du projet.

4.1 Module account

Gère le profil et les paramètres du compte, la gestion d'équipe (invitations, retrait de membres), le changement de mot de passe, le solde de crédits et son historique de consommation, ainsi que les préférences de notification. 17 endpoints sous /account, tous protégés par JWT.

- **bcryptjs** !' réutilisé pour le changement de mot de passe (vérification de l'ancien + hash du nouveau)

- **MailModule** !' envoi des emails d'invitation d'équipe

Tests : `account.controller.spec.ts` — 21 tests sur 8 describe.

4.2 Module api-keys

Permet à un compte de générer des clés API pour intégrer NovaSMS dans ses propres systèmes (limite de 10 clés actives). Une clé n'est jamais stockée en clair : seul son hash SHA-256 est persisté, avec un préfixe/suffixe conservé pour l'affichage (ex. `nvsms_ab12****89ef`).

- **crypto.createHash('sha256')** / **crypto.randomBytes(30)** !' génération et vérification sécurisées des clés, sans dépendance externe

- **ApiKeyGuard + ApiKeyThrottlerGuard** !' authentification par clé (header Authorization ou X-API-Key) et rate-limiting dédié (60 req/min par clé)

- **Permissions granulaires (@RequireApiPermission)** !' `contacts:read`, `contacts:write`, `sms:send`, `email:send`, `campaigns:read`, `balance:read`

- **ApiKeyLogInterceptor** !' journalise chaque appel de l'API publique (code retour, crédits consommés) de façon non bloquante (tap RxJS)

Point d'attention : aucun fichier de test unitaire dédié n'a été trouvé pour ce module — les guards/interceptors ne sont couverts qu'indirectement, s'ils le sont, par les tests du module public-api.

5 — Contacts & Segments — CRM et ciblage

5.1 Rôle

Le module contacts est le plus riche fonctionnellement après campagnes : CRUD, import CSV/Excel massif (deux modes — fichier unique ou upload en chunks), tags, opt-out RGPD, historique, export, validation téléphone en temps réel, et gestion embarquée des segments dynamiques.

5.2 Technologies utilisées et pourquoi

- **exceljs** !' parsing des fichiers Excel importés (plus robuste que des parseurs CSV pour les formats .xlsx réels envoyés par les clients)
- **libphonenumber-js (phone-validator.util.ts)** !' validation et normalisation E.164 des numéros, avec message d'erreur localisé par pays (couverture 245 pays)
- **EventEmitter2** !' émission d'événements métier (contact.added, contact.tag-added, segment.joined) consommés par le module automations, découplant complètement les deux modules
- **BullMQ (queue import-contacts)** !' traite l'import par lots de 500 lignes avec 3 tentatives et backoff exponentiel de 2s, pour supporter des fichiers de plusieurs dizaines de milliers de lignes sans bloquer la requête HTTP
- **@nestjs/schedule (cron quotidien 3h)** !' anonymisation RGPD automatique des contacts désabonnés depuis plus de 30 jours
- **Cache Redis (TTL 300s)** !' évite de recompter à chaque requête le nombre de contacts d'un segment

&™ Mécanisme notable — import volumineux

Deux modes d'import : (a) fichier unique en une requête, (b) upload en chunks streamés (start !' chunk !' complete) avec job BullMQ dédié — objectif observé dans le code : traiter un import de 50 000 lignes en moins de 60 secondes.

Tests : 6 fichiers spec (contacts.controller, contacts.controller.segments, contacts.service, contacts.service.segments — le plus gros fichier de tests du dépôt avec 33 tests —, gdpr-anonymization.service, import.service) — environ 88 tests. Module segments (CRUD dédié, chevauchant fonctionnellement contacts/segments/*) : 15 tests supplémentaires.

5.3 Dette technique identifiée

Le fichier `queues/segment.recalculation.service.ts` importe `InjectQueue/Queue` depuis `@nestjs/bull` (ancienne génération de la librairie), alors que ce paquet n'est pas déclaré dans `apps/backend/package.json` — seul `@nestjs/bullmq` l'est. Cette incohérence entre deux générations de BullMQ dans la même base de code constitue une dette technique à corriger.

6.1 Module campaigns — rôle

Cœur métier de la plateforme : création, édition, duplication, planification et envoi de campagnes SMS et Email, avec A/B testing complet, upload d'images, calcul dynamique du coût d'envoi, et gestion de brouillons.

6.2 Technologies utilisées et pourquoi

- **class-validator/class-transformer** !' validation stricte des DTOs de création de campagne (module écrit plus tôt dans le projet que auth/contacts)
- **BullMQ (queues campaign-dispatch et campaign-schedule)** !' CampaignDispatchProcessor traite les envois par chunks de 500 contacts avec pagination par curseur ; CampaignScheduleWorker (cron chaque minute) scanne les campagnes planifiées échues
- **@aws-sdk/client-s3 + presigner** !' stockage des images de campagne sur S3/MinIO avec URLs présignées, via StorageProviderFactory (fallback stockage local en dev)
- **crypto (HMAC-SHA256)** !' génère un token de tracking unique par envoi, utilisé pour le pixel d'ouverture et la réécriture des liens cliquables

&™ Mécanismes notables

Fenêtre de silence : aucun envoi entre 22h et 8h UTC (report automatique de 30 min)

A/B testing : split configurable, évaluation différée du gagnant (taux de clic puis d'ouverture), diffusion automatique du variant gagnant au reste de l'audience

Déduction de crédit atomique via SQL brut (UPDATE ... WHERE creditBalance >= cost) pour empêcher tout solde négatif en cas d'envois concurrents

Distinction échec permanent (invalid/blacklisted !' pas de retry) vs transitoire (rate-limit/timeout !' retry BullMQ)

Tests : 8 fichiers spec (~94 tests) + 2 suites e2e (campaigns.e2e-spec.ts : 20 tests, campaign-sprint3.e2e-spec.ts) — module le mieux testé en volume du backend avec auth et providers.

6.3 Module templates

CRUD de modèles de messages (email HTML ou SMS texte), avec des templates préconfigurés (isPreset) et prévisualisation par variables. Référencé par le module automations (une automatisation peut pointer vers un templateId). Tests : 17 (templates.controller + templates.service).

6.4 Module track

Trois endpoints publics (sans authentification) : /track/open (pixel d'ouverture), /track/click (redirection trackée), /track/unsubscribe. Chaque appel déclenche une transaction Prisma qui met à jour de façon atomique jusqu'à 5 enregistrements liés (Send, compteurs dénormalisés de Campaign, EngagementHeatmap, ClickHeatmap, Analytic). La sécurité repose sur un token HMAC-SHA256 vérifié en temps constant (crypto.timingSafeEqual) pour empêcher la falsification d'événements. Tests : 13.

7.1 Rôle du module automations

Permet de définir des workflows déclenchés automatiquement par un événement métier (contact ajouté, adhésion à un segment, ouverture ou clic sur une campagne, ajout de tag, anniversaire) pour envoyer un message sans intervention manuelle.

7.2 Technologies utilisées et pourquoi

- **EventEmitter2 (@OnEvent)** !' ContactAddedListener écoute les événements émis par le module contacts/campaigns et déclenche la planification des automatisations correspondantes, sans couplage direct entre modules
- **BullMQ (queue automation-execute)** !' AutomationExecutionProcessor exécute les workflows en file avec gestion de la dernière tentative avant échec définitif

& Point de dette technique identifié

Le module BullMQ est un wrapper de services externes (notamment directement BullMQ) et via BullMQ peut contribuer une divergence potentielle de l'API d'Injection Bull — signe d'un comportement technique complexe plutôt que d'une résolution propre de la cause racine.

Tests : 4 fichiers spec (~32 tests) + 2 suites e2e dédiées (automations-bullmq.e2e-spec.ts, automations-execution.e2e-spec.ts — ce dernier fichier de 14,7 Ko, donc particulièrement détaillé).

7.3 Module queues — workers transverses

Regroupe les processors/workers BullMQ qui ne correspondent pas à un module Nest avec contrôleur propre : campaign.dispatch.queue, campaign.schedule.queue/worker, import.queue (Worker BullMQ natif hors DI Nest, avec stubs dédiés pour l'environnement de test), segment.recalculation.queue/service. Tests : 6 fichiers, ~27 tests.

8.1 Module mobile-money

Initiation, confirmation et suivi des paiements Mobile Money (Wave, Orange Money, MTN MoMo, Moov Money), avec génération de reçu PDF. Les règles métier par opérateur (montants min/max, préfixes téléphoniques valides pour la Côte d'Ivoire/Cameroun) sont codées explicitement (OPERATOR_RULES) — ex. Wave : préfixes 01/05/07/27, plage 500–500 000 FCFA.

- **pdftk** !" génération du reçu de transaction en PDF
- **PaymentProviderFactory** !" bascule simulation !" intégration NovaSend selon la variable MOBILE_MONEY_PROVIDER, sans changement de code métier

Tests : 28 (mobile-money.controller + mobile-money.service).

8.2 Module transactions

Recharge par carte bancaire via Stripe (gestion 3D Secure via requiresAction/clientSecret), historique et reçu. Montant minimum de recharge fixé à 1000 FCFA. Bascule automatique en mode simulation si STRIPE_SECRET_KEY est absent, avec avertissement explicite en log — permet de développer/tester sans compte Stripe réel actif.

Tests : 9 (transactions.controller + transactions.service).

9.1 Module analytics

Fournit une vue d'ensemble des performances (période 7/30/90 jours, comparée à la période précédente), un résumé, l'activité récente, et un rapport détaillé par campagne — consommé à la fois par le Dashboard et la page Analytics du frontend. 4 endpoints, 11 tests.

9.2 Module audit-logs

Consultation paginée, filtrable par type d'action, du journal des actions effectuées sur le compte. Le module lui-même n'écrit pas de logs : il est alimenté directement par Prisma depuis de nombreux autres modules (auth, contacts, campaigns, track...), ce qui en fait un point de convergence transverse plutôt qu'un simple CRUD. 6 tests.

10.1 Module webhooks — double responsabilité

Combine deux fonctions distinctes : (a) réception des webhooks entrants des fournisseurs (Resend, Africa's Talking, Twilio, Stripe) pour mettre à jour le statut des envois, sans guard JWT mais avec vérification manuelle de signature HMAC ; (b) gestion des abonnements webhooks sortants que les clients NovaSMS peuvent configurer pour être notifiés d'événements. Le corps brut de la requête (rawBody) est capturé spécifiquement dans main.ts pour permettre cette vérification de signature. Tests : 40.

10.2 Module public-api — API v1 pour intégration tierce

Expose sous /api/v1 un sous-ensemble de fonctionnalités (solde, CRUD contacts, envoi SMS/Email, liste de campagnes) protégé par ApiKeyGuard + ApiKeyThrottlerGuard, avec journalisation systématique via un interceptor local au module. L'envoi SMS est limité à 500 destinataires par appel selon la documentation Swagger.

10.3 Module mail

Emails transactionnels système (vérification de compte, 2FA, reset mot de passe...), distincts des emails de campagne marketing envoyés via EmailProviderFactory. Utilise resend en production, avec repli automatique sur nodemailer/SMTP local si RESEND_API_KEY est absent. En environnement de test, tous les emails sont redirigés vers une adresse unique (RESEND_TEST_RECIPIENT) pour éviter tout envoi accidentel à de vrais destinataires. 8 tests.

10.4 Module providers — pattern Factory + Strategy

Découple le code métier des fournisseurs externes concrets. Un commentaire du code résume l'intention : « staging et production = exactement le même code, seul .env diffère ». Chaque famille de providers expose une factory avec failover automatique le cas échéant.

Famille	Providers implémentés	Failover
Email	Resend, Brevo, Mock	Oui — Resend !' Brevo
SMS	Twilio, Africa's Talking, NovaSend, Simulation	Oui — configurable
WhatsApp	Twilio WhatsApp Business, Mock	Non — un seul actif
Paieement	Stripe / Simulation (carte), NovaSend / Simulation (Mobile Money)	Non
Stockage	S3 (AWS SDK v3), Local (fallback filesystem)	Non
Push (non branché)	Firebase Cloud Messaging, Mock	Non

& Fait notable

Le sous-module providers pour Firebase Cloud Messaging n'est référencé dans aucun module NodeJS avant 10 ans et n'est pas en sécurité. Il est donc considéré comme potentiellement vulnérable. Il est donc déconseillé d'utiliser ce module dans un environnement de production.

Chaque factory expose une méthode getHealthStatus() consommée par l'endpoint GET /campaigns/providers/health et par la page Intégrations du frontend. Tests : très nombreux fichiers (providers/http-providers.spec.ts, providers/providers.spec.ts, un spec par provider) — plus de 100 tests rien que sur ce module, le plus testé du backend en volume absolu.

11.1 Module common

- **billing.util.ts** !' calcul du coût d'envoi (tarifs par défaut FCFA : SMS 12, Email 2, WhatsApp 35, surchargeables par variables d'environnement), calcul du nombre de segments SMS selon l'encodage GSM7 vs Unicode
- **phone-validator.util.ts** !' wrapper libphonenumber-js réutilisé par contacts et public-api
- **GlobalExceptionHandler** !' uniformise toute réponse d'erreur en JSON et masque les messages techniques (mots-clés prisma/sql/stack) derrière des messages génériques en français
- **RolesGuard + @RequireRoles** !' RBAC transverse (Admin a accès total, sinon vérification stricte du rôle requis)
- **TenantInterceptor + @Tenant()** !' extraction et injection de l'accountId du JWT dans chaque requête (fondement de l'isolation multi-tenant, voir Doc 1 §10)

Tests : global-exception.filter.spec.ts (4 tests) + security.spec.ts (7 tests, probablement centré sur l'isolation tenant et les injections).

11.2 Module prisma

PrismaService étend PrismaClient avec les hooks de cycle de vie NestJS (connexion/déconnexion propre) et est exporté comme module global, utilisé par la quasi-totalité des autres modules.

12 — Synthèse — technologies par module et justification

14

Technologie	Modules utilisateurs	Pourquoi ce choix
BullMQ + Redis	campaigns, contacts, segments, automatisations, queues	Traitement asynchrone fiable avec retries, sans bloquer les requêtes HTTP
libphonenumber-js	contacts, public-api	Validation téléphone internationale (245 pays) sans dépendance à une API tierce navante
speakeasy + bcryptjs	auth	Sécurité des identifiants et 2FA sans dépendance native compléée
Prisma	tous les modules métier	Typage bout-en-bout entre le schéma BDD et le code TypeScript
Factory Pattern (providers)	campaigns, automatisations, mobile-money, transactions	Bascule simulation/production par configuration, sans changement de code
EventEmitter2	contacts, campaigns, automatisations	Découplage entre modules déclencheurs et modules réactifs
crypto (Node natif)	track, api-keys, webhooks	Signatures HMAC et hash sans dépendance externe supplémentaire

Cette synthèse met en évidence une caractéristique structurante du backend : les modules à fort enjeu de fiabilité (envoi de masse, paiement, sécurité) s'appuient systématiquement sur des mécanismes éprouvés (files d'attente, transactions atomiques, signatures cryptographiques) plutôt que sur des solutions ad hoc, ce qui constitue l'un des acquis techniques principaux du stage.